

Regex & TF-IDF

Learning Path

Each block builds the input needed by the next one.

1. **Messy text:** tickets, complaints, logs.
2. **Regex:** extract and normalize fields.
3. **Tokens:** create stable units for counting.
4. **TF-IDF:** combine local focus with global rarity.
5. **Pipeline:** implement the stages and control operational risks.

Why Text Analytics Matters

Most operational signals arrive as text before they become structured data. The goal is not to read everything manually. The goal is to turn language into auditable features.

Source	Typical signal
Tickets	delays, faults, queues
Logs	errors, timeouts, retries
Complaints	refunds, churn, severity
Surveys	free-text satisfaction
Notes	supplier and field context

From Raw Text to Structured Features

Regex is usually the first stabilizing step in the pipeline.
Every transformation should be explainable, repeatable, and testable on examples.

Step	Example
Raw	Printer JAM!!! Ticket=TKT-1042
Extract	ticket_id = TKT-1042
Normalize	printer jam ticket tkt 1042
Tokens	printer, jam, ticket, tkt, 1042

Regex Basics as Building Blocks

A regex is a small grammar for matching text.

Combined pattern:

```
\bTKT-\d{4,6}\b
```

Building block	Pattern	Meaning
Literal	TKT	match these characters
Character class	\d	match one digit
Quantifier	{4,6}	repeat 4 to 6 times
Boundary	\b	do not leak into neighboring text

Character Classes: What Counts as a Match?

Classes define the alphabet that a pattern can consume.

Changing the class changes the evidence collected downstream.

Pattern	Meaning	Example match
<code>\d</code>	one digit	7 in delay 7 days
<code>\d+</code>	one or more digits	1200 in USD 1200
<code>[A-Z]+</code>	uppercase letters	ERR in ERR-404
<code>\w+</code>	word characters	printer

Quantifiers Encode Business Rules

Quantifiers specify how many times the previous unit may repeat. `TKT-\d{4,6}`

`{4,6}` is the rule: valid ticket IDs contain 4 to 6 digits.

When this rule changes, historical data quality changes too.

Document it.

Text	Decision
TKT-1042	accept
TKT-998877	accept
TKT-12	reject
TKT-1042A	reject

Cleaning Example

Raw: "Printer JAM!!! Ticket=TKT-1042"

1. **Extract ID:** TKT-1042 becomes a field.
2. **Lowercase:** Printer JAM becomes printer jam.
3. **Punctuation:** replace punctuation with spaces.
4. **Tokenize:** split into stable terms.

Tokens:

printer | jam | ticket | tkt | 1042

Regex Case: Amount Extraction

Groups separate the currency from the numeric value.

```
(USD|ILS|EUR) \s? (\d+ (?:\.\d{1,2})?)
```

Text: refund USD 120.50 requested

Group 1: currency = USD

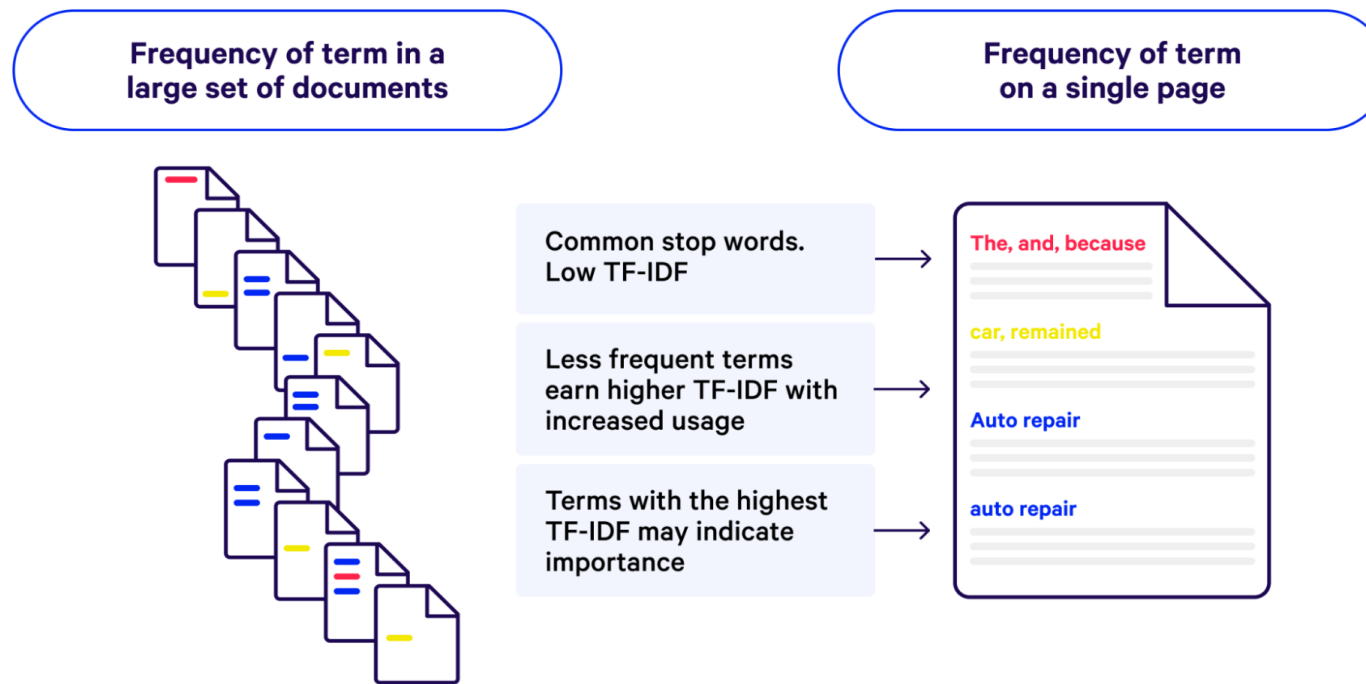
Group 2: amount = 120.50

`(?:\.\d{1,2})?` is a non-capturing group.

It structures the pattern without creating another output field.

TF-IDF Mechanics

Local frequency, global rarity, and worked calculations.



TF-IDF Intuition

A high score needs both local focus and global rarity.

TF-IDF = local focus $\times$ global rarity.

Component	Question	Example
Term frequency	How much does this document talk about the term?	jam = 0.67, printer = 0.33
Inverse document frequency	How rare is this term across the corpus?	outage = 1.10, jam = 0.69

Notation Cheat Sheet

$$tf(t, d) = \frac{count(t, d)}{|d|}$$

$$idf(t) = \log\left(\frac{N}{df(t)}\right) \approx \log\left(\frac{N + 1}{df(t) + 1}\right)$$

$$tfidf(t) = tf(t, d) \cdot idf(t)$$

Symbol	Meaning
t	term after preprocessing
d	one document
N	number of documents
count(t,d)	count of term t in document d
d	number of tokens in document d
df(t)	number of documents containing t

Example

Document 1 data science uses data.

Document 2 data engineering builds pipelines.

Document 3 science uses experiments.

Document 4 engineering uses data pipelines.

Step 1: Counts

Term	S1	S2	S3	S4
data	2	1	0	1
science	1	0	1	0
uses	1	0	1	1
engineering	0	1	0	1
builds	0	1	0	0
pipelines	0	1	0	1
experiments	0	0	1	0

Step 2: Term Frequency (TF)

$$\frac{\text{count of } t \text{ in } d}{\text{number of words in } d}$$

Term	S1	S2	S3	S4
data	0.5	0.25	0	0.25
science	0.25	0	0.333	0
uses	0.25	0	0.333	0.25
engineering	0	0.25	0	0.25
builds	0	0.25	0	0
pipelines	0	0.25	0	0.25
experiments	0	0	0.333	0

Step 3: Inverse Document Frequency (IDF)

$$\log\left(\frac{N + 1}{DF(t) + 1}\right), N = 4$$

Term	df	IDF
data	3	$\log\left(\frac{5}{4}\right) = 0.321$
science	2	0.736
uses	3	0.321
engineering	2	0.736
builds	1	1.321
pipelines	2	0.736
experiments	1	1.321

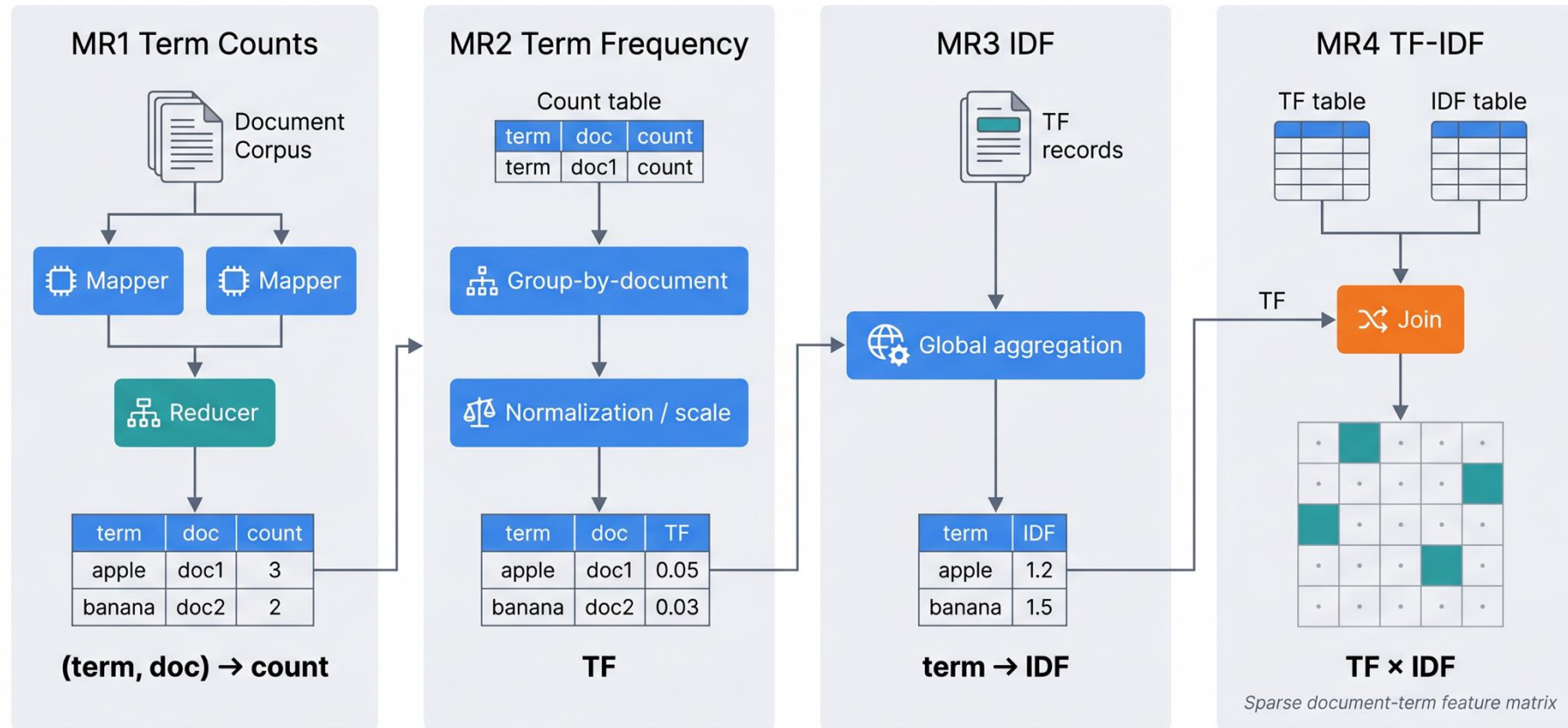
Step 4: TF-IDF

Term	S1	S2	S3	S4
data	0.161	0.080	0	0.080
science	0.184	0	0.245	0
uses	0.080	0	0.107	0.080
engineering	0	0.184	0	0.184
builds	0	0.330	0	0
pipelines	0	0.184	0	0.184
experiments	0	0	0.440	0

Step 4: TF-IDF

Term	S1	S2	S3	S4
data	0.161	0.080	0	0.080
science	0.184	0	0.245	0
uses	0.080	0	0.107	0.080
engineering	0	0.184	0	0.184
builds	0	0.330	0	0
pipelines	0	0.184	0	0.184
experiments	0	0	0.440	0

Distributed TF-IDF



Task 1 Mapper: Word Frequency in Doc

Mapper:

Input: `(docname, contents)`

Output: `((word, docname), 1)`

Example input:

`(D1, "data science uses data.")`

Example output:

`((data, D1), 1), ((science, D1), 1),
((uses, D1), 1), ((data, D1), 1)`

Task 1 Reducer: Sum Word Counts

Reducer:

Input: `((word, docname), [1, 1, ...])`

Output: `((word, docname), n)`

Example shuffle and reduce:

grouped key	values	reducer output
(data, D1)	[1, 1]	((data, D1), 2)
(science, D1)	[1]	((science, D1), 1)
(uses, D1)	[1]	((uses, D1), 1)

Task 2 Mapper: Group by Document

Mapper:

Input: `((word, docname), n)`

Output: `(docname, (word, n))`

Example input from Task 1:

`((data, D1), 2), ((science, D1), 1), ((uses, D1), 1)`

Example output:

`(D1, (data, 2)), (D1, (science, 1)), (D1, (uses, 1))`

Task 2 Reducer: Add Document Length

Reducer:

Input: `(docname, [(word, n), ...])`

Output: `((word, docname), (n, doc_len))`

For D1:

`doc_len(D1) = 2 + 1 + 1 = 4`

Reducer output:

`((data, D1), (2, 4))`

`((science, D1), (1, 4))`

`((uses, D1), (1, 4))`

Task 3 Mapper: Group by Word

Mapper:

Input: `((word, docname), (n, doc_len))`

Output: `(word, (docname, n, doc_len, 1))`

Example mapper output for data:

`(data, (D1, 2, 4, 1))`

`(data, (D2, 1, 4, 1))`

`(data, (D4, 1, 4, 1))`

Each document contributes 1 to `df(data)`.

Task 3 Reducer: Compute Document Frequency

Reducer:

Input: `(word, [(docname, n, doc_len, 1), ...])`

Output: `((word, docname), (n, doc_len, df))`

For data:

`df(data) = 1 + 1 + 1 = 3`

Reducer output:

`((data, D1), (2, 4, 3))`

`((data, D2), (1, 4, 3))`

`((data, D4), (1, 4, 3))`

Task 4 Mapper: Calculate TF-IDF

Mapper:

Input: `((word, docname), (n, doc_len, df))`

Output: `((word, docname), tfidf)`

Example for `(data, D1)`:

$$tf = 2 / 4 = 0.5000$$

$$idf = \ln((4 + 1) / (3 + 1)) = 0.2231$$

$$tfidf = 0.5000 * 0.2231 = 0.1116$$

Mapper output:

`((data, D1), 0.1116)`

Task 4 Reducer: Identity Reducer

Reducer:

Input: `((word, docname), [tfidf])`

Output: `((word, docname), tfidf)`

Example:

Input: `((data, D1), [0.1116])`

Output: `((data, D1), 0.1116)`

The final score is already computed by the mapper.

SQL Implementation: Same Logic

In file “tf-idf.sql”.